

SHADOWINTEL

AI-Powered Criminal Network Intelligence & Investigation Platform

Purpose: A complete build specification and master prompt for Claude to create a polished SIH internal-hackathon prototype from scratch.

1. Product Vision

Build a professional investigator workspace that ingests synthetic FIR/report text, CDR data, financial transactions and location records; extracts entities; resolves duplicate identities; builds a knowledge graph; discovers hidden relationships; detects anomalies; provides timelines and evidence-backed explanations; and generates investigation reports. The system assists investigators and must never claim to determine guilt.

2. Eight Core Modules / Pages

Module	What to build
01 — Command Center	Executive dashboard: active cases, entity counts, relationships, high-priority entities, anomaly alerts, network overview, recent activity and case statistics.
02 — Intelligence Ingestion	Upload/import FIRs and reports, CDR CSVs, transaction CSVs and location CSVs. Show processing stages, extraction counts, validation errors and ingestion history.
03 — Network Explorer	Full-screen interactive knowledge graph. Nodes: people, phones, accounts, locations, vehicles, organizations and events. Edges: called, transferred, visited, met, associated, works-for, owns. Support filtering and 1–3 hop exploration.
04 — Entity Intelligence	Detailed dossier for any entity: identity aliases, priority score, centrality, communities, direct/indirect connections, financial links, communication links, locations, anomalies, timeline and supporting evidence.
05 — Anomaly & Risk Analysis	Detect unusual transactions, communication spikes, location overlaps, new relationships and suspicious temporal sequences. Every alert should show severity, confidence, explanation and evidence.
06 — Investigation Timeline	Unified chronological view of calls, transactions, locations, FIR events and other evidence. Filter by entity/type/date and highlight correlated events.
07 — AI Investigation Assistant	RAG-powered investigator copilot. Answer questions about entities, relationships and evidence with source references. Support natural-language investigation queries and graph-aware answers.
08 — Evidence & Reports	Evidence repository, source preview, SHA-256 integrity hashes, audit history and automated investigation-report generation/export.

3. Recommended Technical Architecture

Frontend: Next.js/React + TypeScript + Tailwind CSS + a component library such as shadcn/ui. Use Cytoscape.js (or equivalent) for the network graph, Recharts/Plotly for analytics, and a map library such as Leaflet for geospatial views.

Backend: Python + FastAPI. Separate services/modules for ingestion, NLP/entity extraction, entity resolution, graph operations, anomaly detection, RAG and report generation.

Graph: Neo4j. Use Cypher for relationship queries and Neo4j Graph Data Science for centrality/community/path algorithms.

Relational/auth/storage: Supabase/PostgreSQL for users, cases, audit records and file metadata; use local files or Supabase Storage for demo evidence.

AI/NLP: spaCy/Transformers or lightweight local models for NER; regex/rules for phones, vehicle numbers, dates and amounts; embeddings + fuzzy matching for entity resolution; RAG over source documents for evidence-backed

answers.

Deployment: Docker Compose for local demo; keep the app capable of running entirely on a laptop using synthetic data.

4. Data Model

Nodes: Person, Phone, BankAccount, Transaction, Location, Vehicle, Organization, Document, Event, Case.

Relationships: CALLED, TRANSFERRED_TO, VISITED, LOCATED_AT, MET, ASSOCIATED_WITH, WORKS_FOR, OWNS, MENTIONED_IN, INVOLVED_IN.

Important metadata: timestamp, source document, confidence, case ID, created_at, evidence ID, relationship confidence and provenance.

5. Demo Dataset — Operation ShadowLink

Create a completely fictional/synthetic investigation dataset. Do not use real criminal records or personal data.

Recommended demo scale: ~50–100 people, 30 phones, 20 accounts, 15 locations, 10 organizations, 300+ CDRs, 100+ transactions, 20–30 reports and several hundred events.

Seed a hidden network into the data but do not explicitly mark the central person. The graph and analytics should discover it. Include at least one bridge entity connecting two communities, one suspicious transaction sequence, one communication burst, one location overlap and one indirect 3-hop relationship.

6. High-Impact Demo Flow

1. Open Command Center and introduce a fictional case: Operation ShadowLink.
2. Upload the synthetic FIR/report, CDR, transaction and location datasets.
3. Show the processing pipeline: parsing → entity extraction → entity resolution → graph update → anomaly analysis.
4. Open Network Explorer and reveal the generated graph and detected communities.
5. Run network analytics and identify a high-priority/bridge entity using centrality metrics. Avoid calling this a 'criminal score'; call it an investigation-priority or suspicious-activity score.
6. Open the entity dossier and show communications, financial links, locations, anomalies and evidence.
7. Use Relationship Finder / natural-language query to discover a 3-hop connection between two entities.
8. Open the Investigation Timeline and show calls, co-location and financial activity clustering in time.
9. Ask the AI Assistant: 'Why are these two entities considered connected?' Return an evidence-backed answer with source references.
10. Open Evidence, verify the SHA-256 hash/audit record, then generate the investigation report.

7. UX / Visual Direction

Make it look like a modern intelligence-analysis platform: dark professional dashboard, high information density without clutter, subtle motion, strong typography, consistent cards, clear severity indicators, polished empty/loading/error states, responsive layout and excellent graph interactions. Avoid cheesy 'hacker' visuals, excessive neon, fake terminal animations or generic AI-chatbot aesthetics.

8. Security & Responsible-AI Requirements

Use synthetic data only. Add role-based access, case-level access checks, audit logging, secure environment variables and evidence hashes. Every AI conclusion should distinguish observed evidence from inference. Clearly label confidence and provide source provenance. Include a disclaimer that the platform assists investigation and does not determine guilt or criminality.

9. Build Strategy for Claude

Do not ask Claude to generate the entire application in one giant response. Have it create the project in vertical slices, run the app/tests after each slice, and preserve a clear architecture. Build in this order:

- Scaffold Next.js/TypeScript frontend, FastAPI backend and shared configuration.
- Create database schemas and seed synthetic Operation ShadowLink data.
- Integrate Neo4j and implement graph CRUD + Cypher queries.
- Build ingestion pipeline and entity extraction.
- Build Network Explorer with real graph data.

- Build Entity Intelligence and Relationship Finder.
- Implement centrality/community analytics and anomaly detection.
- Add Timeline and evidence provenance.
- Integrate RAG assistant with citations to source documents.
- Add report generation, hashes, audit trail and final polish.
- Run end-to-end tests and prepare a single-command demo startup.

10. Master Instruction to Give Claude

You are the lead full-stack engineer, ML engineer and product designer for an SIH internal-hackathon project called SHADOWINTEL – an AI-Powered Criminal Network Intelligence & Investigation Platform.

Build the application completely from scratch as a polished, demo-ready prototype. Do not create a superficial mockup. All important screens must be connected to real seeded synthetic data and the core investigation workflow must work end-to-end.

PRODUCT GOAL The system ingests synthetic FIR/report text, CDR data, financial transactions and location data; extracts entities; resolves duplicate identities; creates a Neo4j knowledge graph; performs graph analytics; detects anomalies; provides a chronological investigation timeline; answers investigator questions using RAG with evidence provenance; and generates investigation reports.

CORE PAGES 1. Command Center 2. Intelligence Ingestion 3. Network Explorer 4. Entity Intelligence 5. Anomaly & Risk Analysis 6. Investigation Timeline 7. AI Investigation Assistant 8. Evidence & Reports

TECHNICAL STACK Frontend: Next.js + TypeScript + Tailwind CSS + shadcn/ui. Graph visualization: Cytoscape.js. Backend: Python + FastAPI. Graph database: Neo4j. Graph analytics: Neo4j Graph Data Science. Relational database/auth/storage metadata: Supabase/PostgreSQL. NLP: spaCy/Transformers plus deterministic regex/rule extraction. Entity resolution: normalized identifiers + fuzzy matching + embeddings where useful. RAG: modular retrieval layer over source documents; keep the LLM provider configurable. Charts: Recharts/Plotly. Maps: Leaflet. Deployment: Docker Compose and local-first development.

DATA MODEL Nodes: Person, Phone, BankAccount, Transaction, Location, Vehicle, Organization, Document, Event, Case. Relationships: CALLED, TRANSFERRED_TO, VISITED, LOCATED_AT, MET, ASSOCIATED_WITH, WORKS_FOR, OWNS, MENTIONED_IN, INVOLVED_IN. Every extracted entity and relationship must retain provenance and confidence metadata.

SYNTHETIC DEMO DATA Create a fictional case called Operation ShadowLink. Generate enough data to make the graph interesting: 50-100 people, 30 phones, 20 bank accounts, 15 locations, 10 organizations, 300+ CDR records, 100+ transactions, 20-30 reports and hundreds of timestamped events. Seed a hidden multi-community network, a bridge entity, suspicious financial activity, a communication burst, a location overlap and a 3-hop relationship. Do not label the hidden answer in the data; the analytics should discover it.

CRITICAL DEMO FEATURES - Entity extraction from reports - Entity resolution/alias matching - Interactive multi-entity knowledge graph - 1-3 hop exploration - Relationship/path finder - Degree/betweenness/PageRank-style metrics where supported - Community detection - Suspicious transaction detection - Communication burst detection - Location overlap detection - Unified timeline - Evidence provenance - AI investigation questions with source-backed answers - SHA-256 evidence hashing and audit events - Investigation report generation

IMPORTANT RESPONSIBLE-AI RULE Never label a person as 'criminal' or make a guilt determination. Use terms such as 'investigation-priority entity', 'suspicious activity', 'potential connection' and 'confidence'. Every AI finding must show its supporting evidence and clearly distinguish evidence from inference.

DESIGN Use a professional intelligence-platform aesthetic. Prioritize information hierarchy, clean typography, excellent graph interactions, subtle animation, strong loading/error/empty states and a responsive layout. Avoid generic SaaS templates, excessive neon/hacker styling and fake AI effects.

DEVELOPMENT METHOD Work in vertical slices. First scaffold the project, then implement the data model and synthetic dataset, then graph infrastructure, then ingestion/NLP, then Network Explorer, then entity intelligence, analytics/anomalies, timeline, RAG assistant, evidence/reporting and finally polish. After each major slice, run tests/build checks and fix errors before proceeding. Keep environment variables documented and never hard-code secrets.

DELIVERABLE A complete runnable repository with: - frontend - backend - database/Neo4j setup - seed scripts - synthetic demo dataset - tests - Docker Compose - README with exact setup/run commands - .env.example - demo walkthrough - architecture documentation

Start by creating the architecture, folder structure, data schemas and implementation plan. Then implement the first vertical slice. Do not stop at design mockups.

11. Can This Be Built for Free?

Yes — for an internal-hackathon prototype, you can keep the software stack at ■0. The important distinction is that 'free' may mean free infrastructure but limited AI/API usage. You can avoid paid model APIs by using local/open-source models for the demo, or keep the LLM layer provider-agnostic.

Neo4j currently offers one free AuraDB instance per account, and as of August 2026 its Aura Graph Analytics is available on AuraDB Free, including the graph-algorithm catalog. ■cite■turn0search5■turn0search13■

Supabase's Free plan currently includes Postgres, authentication, 1 GB storage, 500 MB database size per project and usage quotas; free projects can pause after inactivity. That is more than enough for a synthetic hackathon dataset if you stay within the limits. ■cite■turn0search0■turn0search1■

Claude itself has a Free plan with limited usage, while Claude Pro is currently \$20/month and includes Claude Code. Claude Code usage is subject to plan limits; API-key usage is separately metered/pay-as-you-go. ■cite■turn0search7■turn0search4■turn0search2■

Best ■0 setup: Claude (free/available access) for development assistance + local FastAPI/Next.js + local/open-source NLP/LLM where practical + Neo4j AuraDB Free + Supabase Free + Docker + synthetic data. You can also run the whole demo locally on your laptop, avoiding hosting costs entirely.

What may cost money: heavy cloud LLM usage, paid Claude/API usage, large hosted deployments, a custom domain, or high-volume production traffic. None is necessary for the college demo.